

Programmieren mit System

Wenn du feststeckst, hilft ein klarer Ablauf. Jeder Schritt liefert etwas, mit dem du weiterarbeiten kannst.

1

VERSTEHEN

Was soll passieren?

Formuliere die Aufgabe in eigenen Worten. Kläre Eingabe, gewünschte Ausgabe und offene Fragen.

Notenliste gegeben. Gesucht: der Durchschnitt.

2

KONKRETISIEREN

Wie sieht ein Beispiel aus?

Lege konkrete Eingaben und erwartete Ergebnisse fest, bevor du Code schreibst.

[2, 3, 1, 4]: Summe 10, Anzahl 4, Ergebnis 2.5.

3

ZERLEGEN

Welche kleinen Teile gibt es?

Teile das Ziel in überschaubare Teilprobleme. Benenne, welcher Teil noch unklar ist.

Liste prüfen, addieren, zählen, teilen, ausgeben.

4

PLANEN

Wie löse ich es auf Papier?

Rechne von Hand. Schreibe den Ablauf in Alltagssprache oder Pseudocode auf.

Summe bei 0 starten. Jede Note dazuzählen.

5

IMPLEMENTIEREN

Was ist der kleinste nächste Schritt?

Programmiere einen Teil, den du bereits erklären kannst. Erweitere ihn anschließend.

Erst eine Note addieren, dann alle in einer Schleife.

6

TESTEN

Stimmen Soll und Ist überein?

Sage das Ergebnis voraus, führe den Code aus und prüfe auch einen Randfall.

[3] ergibt 3.0. [] soll „Keine Noten“ ergeben.

7

FEHLER EINGRENZEN

Welche Vermutung kann ich prüfen?

Teste eine Hypothese gezielt, etwa mit print() oder dem Debugger. Ändere eine Sache auf einmal.

Stimmt die Anzahl? Prüfe zuerst len(noten).

8

REFLEKTIEREN

Welcher Gedanke hat geholfen?

Erkläre deinen Lösungsweg. Halte fest, was du beim nächsten Problem wieder anwenden kannst.

„Die Zwischensummen haben den Fehler gezeigt.“

Du darfst zurückgehen: Ein Test kann zeigen, dass du die Aufgabe neu verstehen oder deinen Plan ändern musst.

SO BITTEST DU KONKRET UM HILFE

„Ich möchte X erreichen. Ich habe bisher Y versucht. Ich erwarte A, bekomme aber B. Ich vermute, dass es an C liegt. Bei D komme ich nicht weiter.“

Dein Werkzeugkasten

Wähle die Methode, die zu deiner aktuellen Frage passt. Dein Ziel ist ein kleiner Schritt, der dir neue Klarheit gibt.

Pseudocode

Vor dem Programmieren

Beschreibe die Logik in Alltagssprache. So klärst du den Ablauf, bevor du Python-Syntax brauchst.

„Für jede Note: zur bisherigen Summe addieren.“

Rubber Duck Debugging

Wenn du den Fehler nicht siehst

Erkläre deinen Code einer Person oder einem Gegenstand laut, Zeile für Zeile. Was erwartest du jeweils?

„Hier setze ich die Summe wieder auf 0. Warum?“

Minimal Reproducible Example

Wenn das Problem unübersichtlich ist

Verkleinere Daten und Code, bis nur ein kleines Beispiel übrig bleibt, das denselben Fehler zeigt.

Statt einer ganzen Anwendung: zwei Zahlen und eine Schleife.

Hypothesengetriebenes Debugging

Bei unerwartetem Verhalten

Formuliere eine Vermutung. Plane einen gezielten Test und vergleiche die Beobachtung mit deiner Vorhersage.

„Die Anzahl ist falsch.“ Test: `print(len(noten))`.

Testgetriebenes Denken

Vor und während der Umsetzung

Lege vorher fest, woran du eine korrekte Lösung erkennst. Prüfe Normalfälle und Grenzen.

[1, 6] soll 3.5 ergeben. Für [] klären wir die Ausgabe.

DREI REGELN FÜR JEDES PROGRAMMIERPROBLEM

01 Mach das Problem kleiner.

Suche einen Teil, den du schon lösen oder gezielt ausprobieren kannst.

02 Vermute nicht nur. Experimentiere.

Prüfe deine Hypothese mit möglichst wenig Code und einer klaren Vorhersage.

03 Stelle konkrete Fragen.

Beschreibe, was du erreichen willst, was du beobachtest und welche Stelle unklar ist.

Du musst die Lösung noch nicht kennen, um den nächsten Schritt zu finden.